

EXPLAINABLE MULTI-HORIZON DROPOUT RISK PREDICTION FOR EARLY ADVISOR INTERVENTION IN HIGHER EDUCATION

INDIVIDUAL THESIS

Mahavithana S.M.

(IT22916426)

Individual component: Unified Multi-Horizon GRU with Explainability
for Early Dropout Risk Prediction

Group ID: R26-IT-059

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) Degree in Information Technology

Specialization in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Mahavithana S.M.

Student ID: IT22916426

Signature: Shu . Date: 07.10.2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name			Signature	Date
Supervisor:	Prof.	Anuradha Karunasena		10.07.2026
Co-supervisor:	Ms.	Malithi Nawarathne		10.07.2026

ABSTRACT

Student retention has become a serious issue in higher education globally. Existing early warning systems predict at a single point only and advisors receive only global explanations, therefore advisors cannot decide when to intervene or why a specific student is at risk. This thesis describes the dropout risk prediction component of the R26-IT-059 Academic Burnout and Dropout Risk Detection system: a Unified Multi-Horizon Gated Recurrent Unit (GRU) with a shared encoder, a temporal attention layer and four prediction heads at Weeks 4, 8, 12 and 17, combined with a three-level explainability framework (temporal attention, gradient feature attribution and per-student explanations). The model was trained on the Open University Learning Analytics Dataset (16,950 students) using thirteen weekly engagement and academic features and benchmarked against seven baselines. Advisors could reliably detect risk at Week 8 ($AUC = 0.78$) while 9 to 22 weeks of course time remained, and Week 17 gave the most accurate prediction ($F1 = 0.7227$, $AUC = 0.8831$), a statistically significant improvement over the strongest baseline (McNemar $p = 0.0098$). On a different academic year, Week 17 performance remained strong ($AUC = 0.8275$), although earlier windows fell below 0.75; an exploratory analysis of students taking several modules at once reached $F1 = 0.7464$. The component was implemented as a Flask API and web dashboard and integrated with the team's fusion meta-layer, providing advisors with timely, reason-based alerts.

Keywords: learning analytics, dropout prediction, early warning system, explainable AI, multi-horizon GRU

ACKNOWLEDGEMENTS

I sincerely appreciate and give my gratitude to my supervisor, Prof. Anuradha Karunasena, and my co-supervisor, Ms. Malithi Nawarathne, for their unwavering support and guidance throughout my research journey. I also thank my fellow research group members for their collaboration during the integration process.

TABLE OF CONTENTS

1 INTRODUCTION.....	1
1.1 Overall project context.....	1
1.2 Individual component context.....	1
1.3 Problem statement.....	1
1.4 Research gap	2
1.5 Aim and objectives.....	2
1.6 Research questions or hypotheses.....	3
1.7 Scope limitations and assumptions	3
1.8 Individual contribution.....	3
1.9 Thesis structure	3
2 LITERATURE REVIEW AND RELATED WORK	4
2.1 Review method and scope.....	4
2.2 Domain and theoretical foundations	4
2.3 Related research	4
2.4 Existing systems products and techniques	6
2.5 Gap synthesis and conceptual framework.....	6
3 RESEARCH METHODOLOGY	7
3.1 Research design.....	7
3.2 Data participants or study materials.....	7
3.3 Experimental or study procedure	7
3.4 Measures and evaluation metrics	9
3.5 Data analysis	9
3.6 Ethics privacy security and responsible AI.....	9
3.7 Reproducibility and research artefacts	10

4	PRODUCT AND SOLUTION DESIGN.....	12
4.1	Requirements.....	12
4.2	Component architecture within the complete solution.....	13
4.3	Design alternatives and decisions	13
4.4	Detailed design.....	14
4.5	Standards and professional practice	15
5	IMPLEMENTATION INTEGRATION AND TESTING.....	17
5.1	Development environment and technology stack	17
5.2	Component implementation	17
5.3	Integration with the other components.....	20
5.4	Verification and software testing	22
5.5	Deployment and operation	25
6	RESULTS AND VALIDATION	27
6.1	Research results.....	27
6.2	Product and technical results.....	30
6.3	User or stakeholder validation	30
6.4	Integration results.....	31
6.5	Summary of findings.....	32
7	DISCUSSION	33
7.1	Interpretation of findings.....	33
7.2	Comparison with literature baselines and existing solutions	33
7.3	Research contribution.....	33
7.4	Product contribution.....	33
7.5	Limitations and threats to validity.....	34
7.6	Implications.....	34
8	COMMERCIALISATION AND SUSTAINABILITY	35

8.1 Value proposition and target users.....	35
8.2 Market and competitive position.....	35
8.3 Business or sustainability model.....	36
8.4 Scalability intellectual property and compliance	36
8.5 Contribution to the Sustainable Development Goals	37
9 CONCLUSION AND FUTURE DIRECTIONS	38
9.1 Conclusion.....	38
9.2 Achievement of objectives	38
9.3 Individual research and product contributions	38
9.4 Future directions.....	39
REFERENCES.....	40
GLOSSARY.....	43
APPENDICES.....	44
Appendix A Extended cross-year results	44
Appendix B Academic integrity and AI use disclosure	44

LIST OF FIGURES

Figure 3.1: Label distribution of the working dataset	7
Figure 4.1: Overall architecture of the R26-IT-059 system; this thesis covers the Academic Trend Prediction component (IT22916426)	13
Figure 4.2: Unified Multi-Horizon GRU architecture	15
Figure 5.1: Advisor dashboard showing horizon-wise risk scores, alert levels and top risk drivers for test students	19
Figure 5.2: Advisor dashboard filtered to high-risk students.....	20
Figure 5.3: Sequence of data flow between the prediction component, API, dashboard and meta-layer	22
Figure 6.1: Accuracy versus lead time	27
Figure 6.2: Model comparison	28
Figure 6.3: Temporal attention weights	29
Figure 6.4: Global feature attribution.....	29
Figure 6.5: Per-student explanation for a high-risk student.....	30
Figure 6.6: Count-based vs max-pool aggregation	32

LIST OF TABLES

Table 2.1: Comparison of related work.....	4
Table 3.1: Engineered weekly features	7
Table 3.2: Training configuration of the Unified Multi-Horizon GRU	8
Table 3.3: Key scripts and their purpose.....	10
Table 3.4: Software versions.....	11
Table 4.1: Functional requirements.....	12
Table 4.2: Non-functional requirements	12
Table 4.3: Design decisions	13
Table 4.4: Standards and professional practices applied	15
Table 5.1: Technology stack	17
Table 5.2: REST API endpoints.....	20
Table 5.3: Test cases	22
Table 5.4: Operational workflow	25
Table 6.1: Performance at each intervention window (test set, $n = 2,543$).....	27
Table 6.2: Comparison with baselines (Week 17)	27
Table 6.3: Generalisation validation (Week 17)	31
Table 8.1: Comparison with existing solutions.....	35
Table 9.1: Achievement of objectives.....	38
Table A.1: Cross-year validation results by intervention window.....	44

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AUC	Area Under the ROC Curve
EWS	Early Warning System
F1	Harmonic mean of precision and recall
FNN	Feedforward Neural Network
GRU	Gated Recurrent Unit
LMS	Learning Management System
LSTM	Long Short-Term Memory
OULAD	Open University Learning Analytics Dataset
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
VLE	Virtual Learning Environment
XAI	Explainable Artificial Intelligence

1 INTRODUCTION

1.1 Overall project context

Online and hybrid higher education institutions collect weekly student interaction data through learning management systems, yet student disengagement often goes unnoticed until it is too late to intervene [1], [2]. The R26-IT-059 project, Academic Burnout and Dropout Risk Detection, addresses this problem with an integrated system of four components that together detect students at risk of burnout and dropout and support timely advisor action.

The four components are: Academic Trend Prediction (IT22916426, this thesis), which predicts dropout risk from LMS activity; Emotional Burnout Detection (IT22196392), which analyses academic stress in text; Behavioural Anomaly Detection (IT22215710), which detects unusual patterns in sensor data; and Engagement Classification with the fusion meta-integration layer (IT22253194), which combines all component outputs for the advisor dashboard (Figure 4.1).

1.2 Individual component context

This thesis covers the Academic Trend Prediction component, which predicts dropout risk. It consumes weekly behavioural and academic features derived from LMS logs and produces, for every student, a risk probability at four intervention windows (Weeks 4, 8, 12 and 17) together with explanations of why the student was flagged. Its outputs are passed to the team's fusion meta-layer and displayed in the advisor dashboard.

1.3 Problem statement

Advisors need to know two things before acting: when a dropout alert can be trusted, and why a particular student is at risk. Current early warning systems answer neither question well. Most are trained at a single checkpoint or as separate models per checkpoint, and most explanations are global feature rankings that do not describe an individual student [3], [4]. As a result, alerts arrive late, cannot be prioritised, and lead to generic rather than targeted interventions.

1.4 Research gap

Two independent systematic reviews confirm the gap [2], [5]:

- No unified architecture has empirically established the earliest point at which dropout risk is detectable with enough confidence to justify intervention; prior work trains separate models per horizon [3], [6].
- Explainable AI in learning analytics is mostly limited to global, population-level SHAP summaries; per-student, per-horizon explanations are absent [4], [5].
- No system combines temporal sequence modelling, multi-horizon prediction and per-horizon explainability in a single advisor-facing framework.
- Most OULAD studies evaluate on a single held-out split, leaving generalisation across cohorts and enrolment structures untested.

1.5 Aim and objectives

Aim: to develop and validate an explainable, multi-horizon dropout risk prediction component that tells advisors when to intervene and why a specific student is at risk.

Specific objectives:

- O1 - Engineer weekly engagement and academic features from LMS interaction logs.
- O2 - Design a unified multi-horizon GRU that predicts risk at Weeks 4, 8, 12 and 17 from one shared encoder.
- O3 - Build a three-level explainability framework producing temporal, global and per-student explanations.
- O4 - Benchmark the model against seven baselines and test statistical significance.
- O5 - Validate generalisation across academic years and concurrent-module enrolment.
- O6 - Implement the component as an API and dashboard and integrate it with the team system.

1.6 Research questions or hypotheses

- RQ1 - At which intervention window does dropout risk become reliably detectable ($AUC \geq 0.75$)?
- RQ2 - Does a unified multi-horizon GRU outperform conventional and single-horizon deep learning baselines?
- RQ3 - Which behavioural signals drive dropout risk, and can they be explained per student?
- RQ4 - Does performance generalise to a new academic cohort and to concurrently enrolled students?

1.7 Scope limitations and assumptions

- The component is trained and evaluated on OULAD, a UK distance-learning dataset.
- Only students with at least 17 weeks of activity are included, which introduces a survivorship effect (Section 7.5).
- Withdrawn and Fail outcomes are merged into one at-risk label.
- The component predicts risk and explains it; designing and delivering the intervention itself is out of scope.

1.8 Individual contribution

- Feature engineering pipeline for thirteen weekly features.
- Unified Multi-Horizon GRU with temporal attention and four prediction heads.
- Three-level explainability framework, including a fix to SHAP axis averaging.
- Benchmarking against seven baselines with McNemar significance testing.
- Cross-year and concurrent-student validation protocols with count-based and max-pool aggregation.
- Flask API, advisor dashboard and integration contract with the fusion meta-layer.

1.9 Thesis structure

Chapter 2 reviews related work. Chapter 3 describes the methodology. Chapter 4 presents the solution design and Chapter 5 its implementation, integration and

testing. Chapter 6 reports results, Chapter 7 discusses them, Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

Relevant literature was searched in Google Scholar using keywords such as ‘dropout prediction’, ‘early warning system’, ‘learning analytics’, ‘explainable AI’ and ‘OULAD’. Studies published between 2017 and 2025 were considered, with priority given to peer-reviewed journals indexed in Scimago. After screening titles and abstracts, 23 studies were selected for detailed review. They fall into five themes: early warning systems, machine learning-based dropout prediction, deep and temporal learning models, explainable AI, and student burnout and stress. Two systematic reviews [2], [5] were also used to confirm the research gap.

2.2 Domain and theoretical foundations

Early warning systems translate predictive models into institutional action [2]. Recurrent neural networks such as LSTM and GRU are well suited to weekly LMS data because they model how engagement evolves over time; deep sequential models consistently outperform classical methods on VLE-rich data [1], [2], [7]. Explainability methods such as SHAP and gradient attribution make these models interpretable to non-specialists [4].

The trade-off between prediction earliness and accuracy is central: early predictions leave more time to act but are less accurate. Jimenez et al. formalised this as a Pareto problem [6].

2.3 Related research

Research on student dropout prediction has grown substantially over the past decade. The studies reviewed fall into five groups: traditional and ensemble machine learning, deep sequential models, explainable AI, temporal prediction horizons, and burnout and wellbeing analytics. Each group is discussed below, and the main studies are compared in Table 2.1.

2.3.1 Traditional and ensemble machine learning

Early studies showed that dropout can be predicted from LMS activity logs, demographic information and academic performance. Akçapınar et al. were among

the first to test prediction at several points in a course, applying a k-nearest neighbour classifier at Weeks 3, 6, 9, 12 and 14; accuracy rose from 74% at Week 3 to 89% at Week 14 [3]. This confirmed that predictive performance improves as the observation window grows, which motivates the multi-horizon design of this thesis. However, the five checkpoints were five independent experiments, each trained from scratch with no shared temporal memory, so the question of which week offers the best balance between accuracy and remaining intervention time was left open.

Later studies applied more powerful ensemble and boosting methods. Villar and de Andrade compared decision trees, SVM, random forest and several boosting algorithms and found LightGBM and CatBoost tuned with Optuna performed best [8]. Rabelo and Zarate combined logistic regression, a neural network and a decision tree into an ensemble that classified 89% of students correctly [9]. Rebelo Marcolino et al. trained CatBoost on Moodle logs with ADASYN balancing and NSGA-II hyperparameter optimisation, reaching an average F1 of about 0.8 across weeks [10]. Vaarma and Li showed that the importance of LMS, transcript and demographic predictors changes from month to month after enrolment [11], and Chang et al. combined LMS-based prediction for 2,856 students with personalised interventions [12]. Together these studies confirm that class balancing and careful optimisation improve the identification of at-risk students. However, they rely on static features extracted at one point in time, so they cannot model how a student's behaviour evolves or predict risk at several future horizons at once.

2.3.2 Deep learning and sequential modelling

Because learning behaviour unfolds over time, recent work has adopted deep models that read sequences of weekly interactions. Waheed et al. applied an LSTM to 22,437 OULAD students and reached 84.57% accuracy; using only the first five weeks of activity, it still achieved a diagnostic accuracy of up to 71% [1]. Torkhani and Rezgui compared machine learning and deep learning models on OULAD and found LSTM the most accurate at 83.41% [7]. Other architectures have also been explored: CNN autoencoders combined with LSTM [23], hybrid RNN, LSTM and machine-learning pipelines [24], temporal graph networks [22], attention-based Kolmogorov–

Arnold networks [17], CNN–BiGRU hybrids [18], LSTM–Transformer models [15] and multi-category neural forecasting at the early stages of a course [16].

Two systematic reviews summarise this body of work. Alnasyan et al. reviewed 46 studies published between 2019 and 2023 and concluded that deep learning consistently outperforms traditional methods on virtual learning environment data, with behavioural engagement features the strongest predictors [2]. Jin et al. reviewed 17 OULAD studies published between 2017 and 2024 and grouped them into performance prediction, at-risk identification and engagement prediction [26]. Despite these advances, most deep learning studies still evaluate prediction at a single predefined time point or at a few independent checkpoints, giving little insight into how performance changes across intervention horizons.

2.3.3 Explainable artificial intelligence in educational prediction

As models become more complex, explainability becomes essential, because predictions must support decisions made by human advisors. Nagy and Molontay argued that instance-level explanations of why an individual student was flagged are far more actionable than population-level feature rankings, and they evaluated SHAP, LIME, permutation importance and partial dependence plots with higher-education stakeholders [4]. Several studies have added explanation methods to dropout models: SHAP-based global importance with an LSTM [1], combined SHAP and LIME analysis of a hybrid logistic-regression and neural-network model [13], SHAP with an attention-based network [17], and attention alongside SHAP in an LSTM–Transformer, where attention served an architectural rather than an explanatory role [15]. The comprehensive review by Boujmiraz et al. confirmed that XAI adoption in educational analytics remains limited, and that no study provides temporal explanations showing how important features change across a student's learning journey, nor personalised explanations for several prediction horizons at once [5]. This limitation motivates the per-student, per-horizon explainability framework of this thesis.

2.3.4 Temporal prediction horizons and early intervention

A recurring question in student-success research is how early reliable predictions can be made while leaving enough time for intervention. Akçapınar et al. showed that

accuracy improves across successive weekly checkpoints [3]. Jimenez et al. formalised the problem as a multi-objective optimisation, using an evolutionary algorithm to build a Pareto front that balances earliness against accuracy; this is the conceptually closest existing work to the research question of this thesis [6]. Waheed et al. and Junejo et al. further showed that early-semester data already carries a meaningful predictive signal, with strong performance possible at 20% of course completion [1], [16]. However, each of these studies either builds separate models for each horizon or evaluates a small number of checkpoints independently. The reviews of Alnasyan et al. and Boujmiraz et al. found no study that trains a unified multi-horizon architecture to identify an optimal intervention window empirically [2], [5]. This gap is the primary motivation for the Unified Multi-Horizon GRU.

2.3.5 Burnout prediction and student wellbeing analytics

Beyond academic dropout, recent research has examined student burnout as a related dimension of risk. Yeskuatov et al. built machine-learning models for the three burnout dimensions of exhaustion, cynicism and reduced professional efficacy using only features engineered from university administrative records; logistic regression achieved an F1 of 68.4% for exhaustion [25]. This shows that burnout can be detected automatically from passively collected institutional data, which matches the design principle of this thesis. However, the study used a cross-sectional design without temporal sequence modelling or multi-horizon prediction. Temporal and multi-horizon approaches therefore remain under-explored in the burnout literature, which supports the LMS-based, multi-horizon approach adopted here and its integration with the emotional and behavioural components of R26-IT-059.

Table 2.1: Comparison of related work

Study	Multi-horizon (unified)	Sequential model	Per-student XAI	Multi-protocol validation
Akçapınar et al. [3]	No (separate models)	No	No	No
Jimenez et al. [6]	Optimised, not unified	No	No	No

Waheed et al. [1]	No	Yes (LSTM)	No	No
Nagy & Molontay [4]	No	No	Yes (SHAP, LIME)	No
Mustofa et al. [13]	No	No	Yes (SHAP, LIME)	No
Torkhani & Rezgui [7]	No	Yes (LSTM)	No	No
Ofori & Dake [15]	No	Yes (LSTM-Transformer)	Global only (SHAP)	No
Junejo et al. [16]	Partial (early stage only)	No	No	No
Alnasyan et al. [17]	No	Yes (self-attention)	No	No
Adefemi et al. [18]	No	Yes (CNN-BiGRU)	No	No
This work	Yes	Yes (GRU + attention)	Yes, per horizon	Yes (3 protocols)

2.4 Existing systems products and techniques

Commercial and institutional tools such as LMS-embedded analytics dashboards provide risk flags, but typically rely on rule-based thresholds or single-point predictions and offer limited explanation of individual risk.

Moodle's built-in analytics include a 'Students at risk of dropping out' model that flags students as at risk or not at risk, without explaining individual predictions [19]. The Open University's OU Analyse provides teachers with weekly predictions of each student's risk of failing through interactive dashboards [20]. Commercial platforms such as EAB Navigate and Civitas Learning offer proprietary risk scores.

None of these systems identifies when an alert first becomes reliable or provides per-horizon explanations for each student; a detailed comparison is given in Table 8.1.

2.5 Gap synthesis and conceptual framework

Three gaps remain: (1) single-point or separately trained horizons, (2) global-only explanations, and (3) no framework combining sequence modelling, multi-horizon prediction and per-horizon explainability. The conceptual framework of this thesis links weekly behavioural signals → shared temporal representation → horizon-specific risk → explanation → advisor action.

3 RESEARCH METHODOLOGY

3.1 Research design

The study follows a quantitative, experimental design: a predictive model is developed and compared with baselines on a public dataset under identical conditions, followed by additional validation protocols to test generalisation.

3.2 Data participants or study materials

OULAD [14] contains 32,593 students across 22 module presentations with over 10.6 million VLE interaction records. Students with a final result of Withdrawn or Fail were labelled at-risk (1). After filtering to students with at least 17 weeks of data, 16,950 students remained, of whom 31.4% were at-risk (Figure 3.1).

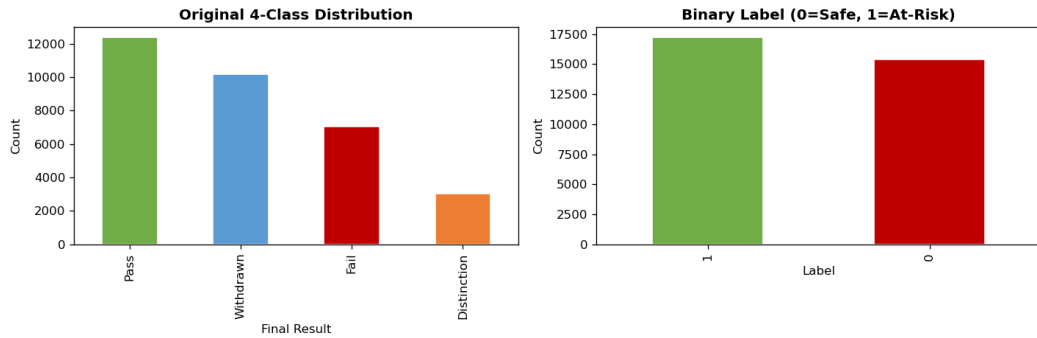


Figure 3.1: Label distribution of the working dataset

3.3 Experimental or study procedure

Thirteen features were computed weekly per student (Table 3.1) and z-score normalised using training-set statistics. Data was split 70/15/15 (stratified) into training, validation and test sets. All four prediction heads were trained jointly with averaged binary cross-entropy and a class weight of 2.18 for the at-risk class; per-window decision thresholds were tuned on the validation set. Seven baselines were trained on identical data.

Table 3.1: Engineered weekly features

ID	Feature	Type
F1	login_count	Engagement

F2	vle_clicks	Engagement
F3	avg_score	Academic
F4	num_submitted	Academic
F5	on_time_rate	Engagement
F6	inactive_days	Engagement
F7	score_change	Academic
F8	running_avg	Academic
F9	score_variance	Academic
F10	min_score	Academic
F11	max_score	Academic
F12	late_count	Engagement
F13	weeks_since_active	Engagement

The training configuration of the proposed model is summarised in Table 3.2.

Table 3.2: Training configuration of the Unified Multi-Horizon GRU

Parameter	Value
Framework	PyTorch
Hardware	Apple Silicon GPU (MPS)
Optimiser	Adam (weight decay 1×10^{-5})
Learning rate	0.0001
Learning rate scheduler	ReduceLROnPlateau (factor 0.5, patience 10)
Batch size	64

Maximum epochs	500
Early stopping	Patience 50 epochs (stopped at epoch 137; best epoch 87)
Loss function	Binary cross-entropy with logits, class weight 2.18
GRU layers / hidden size	2 / 256
Dropout	0.3
Gradient clipping	Max norm 1.0
Trainable parameters	767,493
Random seed	42

3.4 Measures and evaluation metrics

F1-score, AUC-ROC, precision and recall were reported at each window. F1 balances missed at-risk students against unnecessary interventions; AUC measures ranking quality independent of threshold. $AUC \geq 0.75$ was treated as the operationally useful threshold.

3.5 Data analysis

Model comparison used McNemar's test on paired test-set predictions. Explainability was analysed at three levels: population-averaged attention weights, gradient attribution normalised to 100%, and per-student attribution and attention profiles. Generalisation was tested by (a) training on 2013 and testing on 2014, and (b) aggregating per-module risks for 839 concurrently enrolled students using count-based and max-pool strategies.

3.6 Ethics privacy security and responsible AI

No formal ethics approval was required, as the study used only the publicly available, fully anonymised OULAD dataset (released under a CC-BY 4.0 licence) and involved no direct interaction with human participants. The following responsible-AI measures were applied:

- No personal identifiers were used; students are referenced only by anonymised IDs.
- Predictions are decision support for advisors, not automatic decisions about students.
- Explanations are provided so advisors can question and override alerts.
- Risk of bias: the model may reflect patterns of the source institution; fairness across demographic groups should be audited before deployment.
- Dashboard access should be role-based and risk labels kept confidential.

3.7 Reproducibility and research artefacts

Code, notebooks and README files are available at github.com/Sha20000/R26-IT-059-Burnout-Prediction. The dataset is publicly available from the Open University.

The main scripts used in this study are listed in Table 3.3, and the software versions are given in Table 3.4. All experiments used a fixed random seed (42) and the same stratified 70/15/15 split to ensure reproducibility.

Table 3.3: Key scripts and their purpose

Script	Purpose
data_loader.py	Loads OULAD, builds the 13 weekly features, normalises them and creates the stratified split
model_1_random_forest.py	Random Forest baseline
model_2_gradient_boosting.py	Gradient Boosting baseline
model_3_xg_boost.py	XGBoost baseline
model_4_logistic_regression.py	Logistic Regression baseline
model_5_lstm.py	Simple LSTM baseline
model_6_gru.py	Single-task GRU baseline
model_7_bilstm.py	BiLSTM baseline
model_8_unified_gru.py	Proposed Unified Multi-Horizon GRU,

	explainability and prediction export
cross_year_validation.py	Cross-year (2013 → 2014) validation
mcnemar_significance_test.py	McNemar statistical significance test
app_api.py	Flask REST API serving predictions to the dashboard

Table 3.4: Software versions

Library	Version
Python	3.13.5
PyTorch	2.11.0
scikit-learn	1.8.0
SHAP	0.52.0
pandas	3.0.2
NumPy	2.4.4
Flask	3.1.3

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The functional requirements of the component are listed in Table 4.1 and the non-functional requirements in Table 4.2.

Table 4.1: Functional requirements

ID	Requirement
FR1	Ingest weekly LMS activity and assessment data
FR2	Compute 13 weekly features per student
FR3	Predict dropout risk at Weeks 4, 8, 12, 17
FR4	Generate per-student explanations
FR5	Expose predictions via REST API
FR6	Display risk and explanations in advisor dashboard
FR7	Aggregate risk for concurrently enrolled students
FR8	Send outputs to the fusion meta-layer

Table 4.2: Non-functional requirements

ID	Requirement
NFR1	Accuracy: $AUC \geq 0.75$ by Week 8
NFR2	Interpretability: every alert has a reason
NFR3	Performance: API response within 0.5 s
NFR4	Privacy: anonymised IDs, role-based

	access
NFR5	Maintainability: documented, modular code

4.2 Component architecture within the complete solution

The component sits between the data layer and the team's fusion meta-layer. Weekly features enter the prediction service, which returns horizon-wise risk and explanations to the meta-layer and the advisor dashboard.

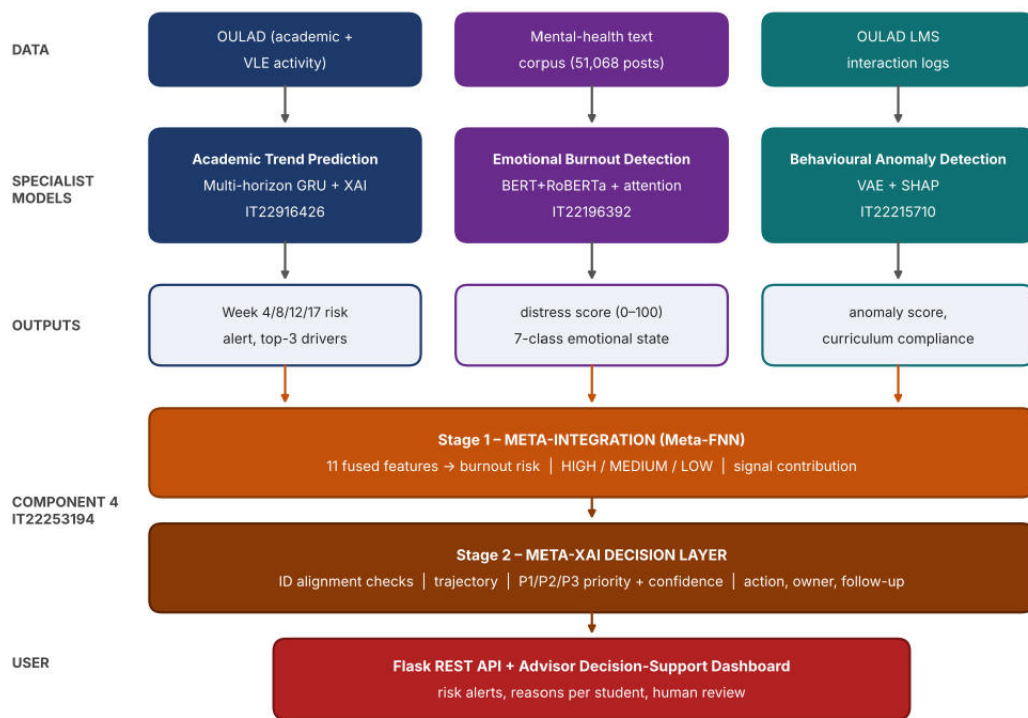


Figure 4.1: Overall architecture of the R26-IT-059 system; this thesis covers the Academic Trend Prediction component (IT22916426)

4.3 Design alternatives and decisions

The main design alternatives considered and the reasons for each choice are summarised in Table 4.3.

Table 4.3: Design decisions

Decision	Alternatives	Choice and reason

Sequence model	LSTM, BiLSTM, Transformer	GRU - fewer parameters, comparable accuracy, faster training
Horizons	Separate models per week	Unified shared encoder - one model, consistent representation
Explainability	SHAP only	Attention + gradients + per-student - covers when, what, why
Concurrent aggregation	Count-based	Max-pool - avoids recall collapse
API	FastAPI, Django	Flask - lightweight, simple integration

4.4 Detailed design

The model has three parts (Figure 4.2). A two-layer GRU encoder (hidden size 256, dropout 0.3) reads the 17-week sequence. A soft temporal attention layer weights the weekly hidden states. Four heads at Weeks 4, 8, 12 and 17 each apply $\text{Linear}(256 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(32 \rightarrow 1)$ with a sigmoid at inference.

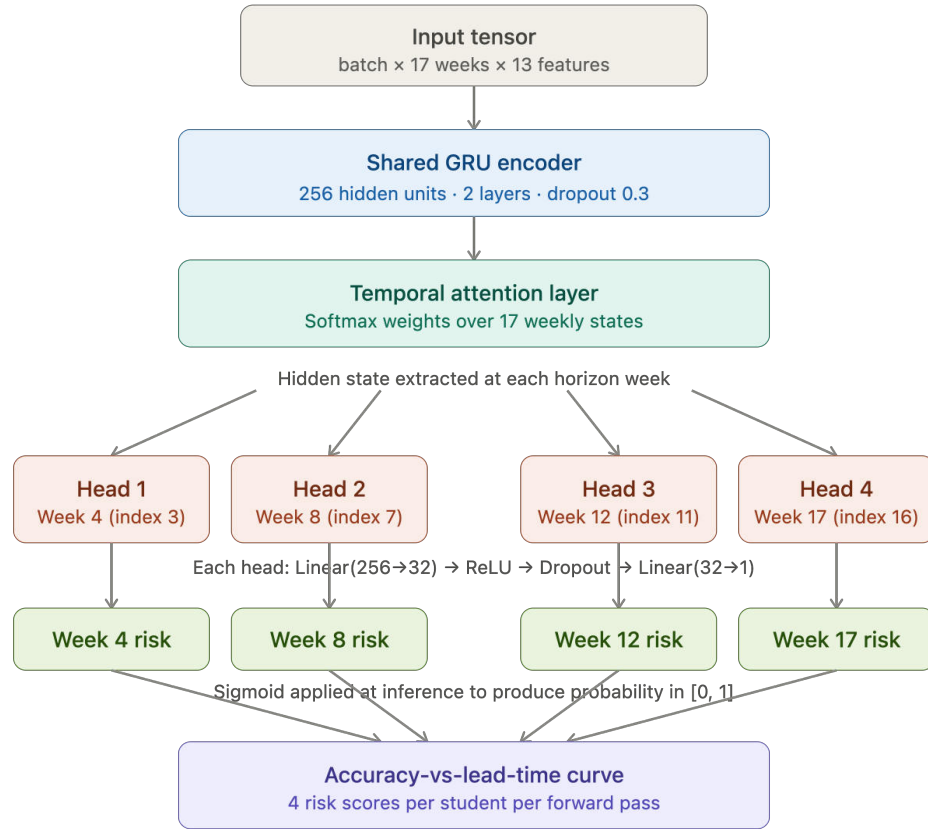


Figure 4.2: Unified Multi-Horizon GRU architecture

The explainability layer produces: Level 1 - population-averaged attention (when); Level 2 - gradient attribution (what); Level 3 - per-student attribution and attention profile (why this student).

4.5 Standards and professional practice

Professional software engineering practices were followed throughout development to keep the component maintainable, reproducible and trustworthy. The standards and practices applied are summarised in Table 4.4.

Table 4.4: Standards and professional practices applied

Practice	How it was applied
Version control	Source code managed with Git and hosted on GitHub (github.com/Sha20000/R26-IT-059-Burnout-Prediction)
REST API design	Flask backend with HTTP GET endpoints, JSON responses and standard status codes (200, 404, 500)
Documentation	Separate README files for the backend and frontend, plus a setup guide (FLASK_SETUP.md)
Reproducibility	Fixed random seed (42), fixed stratified 70/15/15 split and recorded software versions (Section 3.7)
Structured testing	Test cases documented with ID, description, input, expected result and status (Section 5.4)
Responsible AI	Predictions presented as decision support with explanations, following transparency and human-oversight principles (Section 3.6)
Referencing	All sources cited using the IEEE referencing style

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The technologies used to implement the component are listed in Table 5.1.

Table 5.1: Technology stack

Layer	Technology	Purpose
Language	Python 3.13.5	Model and API
Deep learning	PyTorch 2.11.0	GRU model
Explainability	SHAP 0.52.0	Feature attribution
Data	pandas, NumPy, scikit-learn	Features, baselines, metrics
Baselines	XGBoost, scikit-learn	Comparison models
API	Flask	Prediction service
Frontend	HTML, CSS, JavaScript	Advisor dashboard
Tools	Jupyter, Anaconda, GitHub	Development

5.2 Component implementation

The component was implemented in Python using PyTorch. Its main parts are the model class, the joint multi-horizon training loop, per-horizon threshold tuning, and a Flask API that serves predictions to the dashboard.

Listing 5.1: Forward pass of the Unified Multi-Horizon GRU



The shared GRU encoder reads the 17-week sequence, and each of the four heads predicts risk from the hidden state at its own horizon, so no future weeks are used.

Listing 5.2: Joint multi-horizon training step

```

1  for X_batch, y_batch in train_loader:
2
3      optimizer.zero_grad()
4
5      #Forward pass - get 4 predictions
6      predictions, _ = model(X_batch)
7
8      #Calculate loss for each head
9      #All heads predict same label
10
11      total_loss = 0
12      for pred in predictions:
13          loss = criterion(
14              pred.squeeze(-1), y_batch
15          )
16          total_loss += loss
17
18      #Average loss across 4 heads
19      # You, 5 months ago * Average loss across 4 heads ...
20
21      total_loss = total_loss / len(predictions)
22
23      total_loss.backward()
24
25      #Gradient clipping to prevent exploding gradients
26      torch.nn.utils.clip_grad_norm_(
27          model.parameters(), max_norm=1.0
28      )
29
30      optimizer.step()
31      epoch_loss += total_loss.item()
32
33      avg_loss = epoch_loss / len(train_loader)
34      train_losses.append(avg_loss)
35

```

All four heads are trained together; their binary cross-entropy losses are averaged, gradients are clipped for stability, and early stopping on validation F1 selects the best model.

Listing 5.3: Per-horizon threshold tuning on the validation set

```

1  val_pred_i = torch.sigmoid(
2      model(X_val_t)[0][i]).squeeze().cpu().detach().numpy()
3
4  best_thresh = 0.5
5  best_f1_v = 0
6  for t in np.arange(0.2, 0.71, 0.05):
7      p = (val_pred_i > t).astype(int)
8      f = f1_score(data['y_val'], p,
9                  zero_division=0)
10
11      if f > best_f1_v:
12          best_f1_v = f
13          best_thresh = t
14
15  preds = (probs > best_thresh).astype(int)
16  f1 = f1_score(y_true, preds)
17  auc = roc_auc_score(y_true, probs)
18  pre = precision_score(y_true, preds,
19                       zero_division=0)
20  rec = recall_score(y_true, preds, zero_division=0)
21
22  print(f"label:<12>| {f1:>6.4f} | ")
23

```

Each horizon uses its own decision threshold, selected on validation data (Week 4: 0.45, Week 8: 0.50, Week 12: 0.50, Week 17: 0.55).

Listing 5.4: Flask endpoint returning one student's prediction with input validation

```

144 @app.route('/api/test-students/<student_id>', methods=['GET'])
145 def get_test_student(student_id):
146     """Get single test student by ID"""
147     if TEST_STUDENTS_DF is None:
148         return jsonify({"error": "Test data not loaded"}), 500
149
150     # Try to find by student_id or index
151     student = TEST_STUDENTS_DF[TEST_STUDENTS_DF['student_id'] == student_id]
152
153     if student.empty:
154         # Try by index
155         try:
156             idx = int(student_id)
157             student = TEST_STUDENTS_DF.iloc[[idx]]
158         except:
159             return jsonify({"error": "Student not found"}), 404
160
161     if student.empty:
162         return jsonify({"error": "Student not found"}), 404
163
164     return jsonify(student.iloc[0].to_dict())

```



Listing 5.4 shows the endpoint that returns a single student's horizon-wise risk scores, alert level and top risk drivers. The endpoint validates the request and returns HTTP 500 if predictions are not loaded and HTTP 404 if the student index is out of range, while `clean_row()` converts the record into JSON-safe values.

The advisor dashboard (Figure 5.1) loads the predictions from the API and shows each student's horizon-wise risk scores, alert level and top risk drivers.

Academic Burnout & Dropout Risk Detection
 Test Students (2543) + PP2 Concurrent (839) · API-Enabled Backend

[R26-IT-059](#)
[Live API](#)
[3.85/5.00](#)

[Test Students](#)
[PP2 Concurrent](#)
[Model Metrics](#)
[Meta-FNN Risk](#)
[Emotional Analysis](#)
[Academic Risk](#)
[Behavioral Anomaly](#)
[SLIIT Synthetic](#)

FILTER BY ALERT LEVEL: All Students
 SEARCH STUDENT ID: e.g., OULAD_TEST_0007
 LIMIT RESULTS: All

Student ID	Overall Risk Score	Week 4 Risk	Week 8 Risk	Week 12 Risk	Week 17 Risk	Alert Level	Top Feature	Importance %	Critical Week	Outcome
OULAD_TEST_0000	25.6%	50.2%	19.6%	37.0%	25.6%	LOW	F1_login_count	17.7%	Week 16	✓ PASSED
OULAD_TEST_0001	3.3%	23.9%	15.1%	7.8%	3.3%	LOW	F13_weeks_since_active	12.9%	Week 8	✓ PASSED
OULAD_TEST_0002	6.4%	47.4%	35.3%	23.7%	6.4%	LOW	F1_login_count	17.6%	Week 8	✓ PASSED
OULAD_TEST_0003	39.2%	71.2%	53.7%	48.0%	39.2%	LOW	F1_login_count	16.3%	Week 5	✓ PASSED
OULAD_TEST_0004	10.8%	33.9%	16.6%	9.8%	10.8%	LOW	F6_inactive_days	15.8%	Week 1	✗ DROPOUT
OULAD_TEST_0005	25.1%	67.4%	50.7%	23.7%	25.1%	LOW	F1_login_count	19.2%	Week 11	✗ DROPOUT
OULAD_TEST_0006	22.2%	47.7%	47.3%	38.9%	22.2%	LOW	F6_inactive_days	14.9%	Week 7	✓ PASSED
OULAD_TEST_0007	99.2%	44.2%	74.4%	83.6%	99.2%	HIGH	F1_login_count	29.0%	Week 14	✗ DROPOUT
OULAD_TEST_0008	39.8%	77.8%	80.5%	71.9%	39.8%	LOW	F1_login_count	14.8%	Week 17	✓ PASSED
OULAD_TEST_0009	41.4%	29.7%	18.7%	25.4%	41.4%	MEDIUM	F1_login_count	21.8%	Week 1	✗ DROPOUT
OULAD_TEST_0010	86.5%	52.5%	34.0%	39.6%	86.5%	HIGH	F1_login_count	21.2%	Week 10	✗ DROPOUT
OULAD_TEST_0011	99.9%	84.0%	94.1%	98.0%	99.9%	HIGH	F1_login_count	22.4%	Week 9	✗ DROPOUT

Figure 5.1: Advisor dashboard showing horizon-wise risk scores, alert levels and top risk drivers for test students

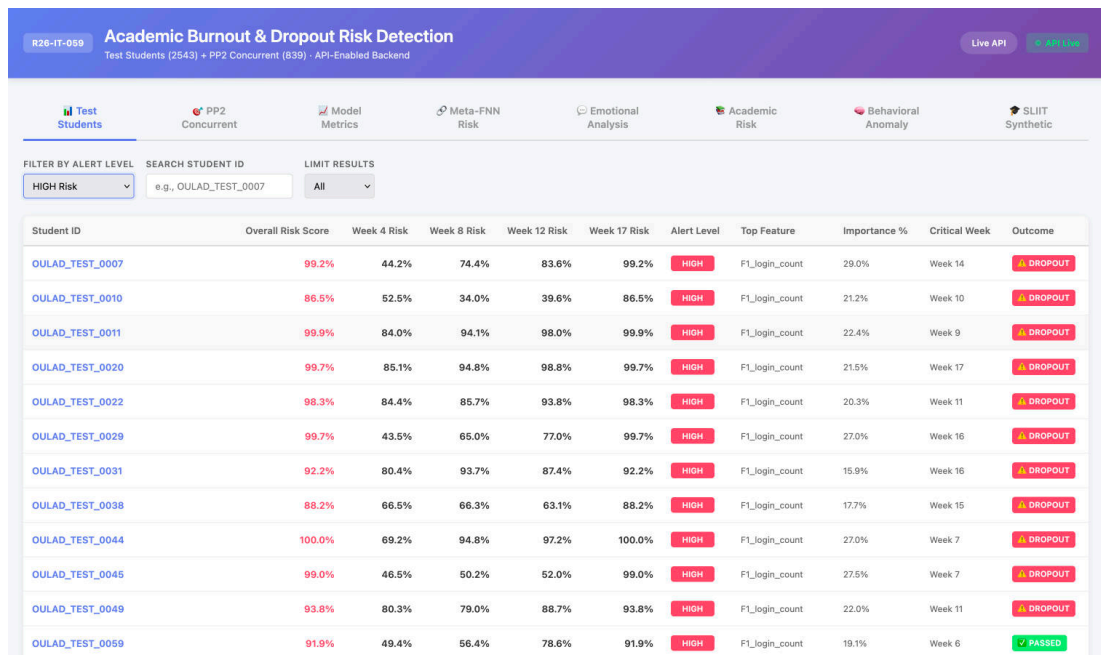


Figure 5.2: Advisor dashboard filtered to high-risk students

Advisors can filter students by alert level; Figure 5.2 shows the high-risk view, listing students with their horizon-wise risk and main risk driver.

5.3 Integration with the other components

The component integrates with the rest of the R26-IT-059 system through file-based and API-based contracts. The batch prediction step exports risk scores for all 2,543 test students as CSV and JSON files. These files are (i) consumed by the fusion meta-layer (IT22253194), which combines the academic risk with the other components' outputs, and (ii) loaded by a Flask REST API that serves them to the advisor dashboard. The available endpoints are listed in Table 5.2, the agreed data contract is shown in Listing 5.5, and the data flow is illustrated in Figure 5.3.

Table 5.2: REST API endpoints

Endpoint	Purpose
GET /health	API status and number of loaded predictions

GET /students	All predictions (filter: ?alert=HIGH/MEDIUM/LOW)
GET /students/<idx>	One student by index (404 if out of range)
GET /search?id=<student_id>	One student by ID (400 if ID missing, 404 if not found)
GET /summary	Alert-level counts, F1, AUC and optimal horizon

Listing 5.5: Data contract: example JSON record for one student

```

1  {
2    "student_index":0,
3    "student_id":"OULAD_TEST_0000",
4    "actual_label":0,
5    "academic_risk":0.2561,
6    "week4_risk":0.5822,
7    "week8_risk":0.1955,
8    "week12_risk":0.3702,
9    "week17_risk":0.2561,
10   "alert_level":"LOW",
11   "top_feature_1":"F1_login_count",
12   "top_feature_1_pct":17.7,
13   "top_feature_2":"F6_inactive_days",
14   "top_feature_2_pct":14.7,
15   "top_feature_3":"F4_num_submitted",
16   "top_feature_3_pct":14.6,
17   "most_important_week":16,
18   "main_reason":"F1 login count (17.7%) and F6 inactive days (14.7%)"
19 },
20
21 {
22   "student_index":1,
23   "student_id":"OULAD_TEST_0001",
24   "actual_label":0,
25   "academic_risk":0.0329,
26   "week4_risk":0.2391,
27   "week8_risk":0.1514,
28   "week12_risk":0.0785,
29   "week17_risk":0.0329,
30   "alert_level":"LOW",

```

Each record provides horizon-wise risk probabilities, the final academic risk (Week 17), a display alert level and the three strongest risk drivers with their contribution percentages, allowing the meta-layer and dashboard to consume a consistent format.

Figure 5.3 shows the sequence of data flow. The batch prediction script (`model_8_unified_gru.py`) writes the horizon-wise risks, alert level and top three risk drivers to the CSV and JSON prediction files (step 1), which the Flask API loads at start-up (step 2). The dashboard requests the high-risk list through GET `/students?alert=HIGH` (steps 3–4) and a single student's record through GET `/students/<idx>` or `/search?id=`, receiving either the record with its risk drivers or an

HTTP 404 response (steps 5–6). The academic risk scores are also passed to the Meta-FNN layer (IT22253194) through the CSV/JSON data contract (step 7), which returns the fused multi-component risk to the dashboard (step 8).

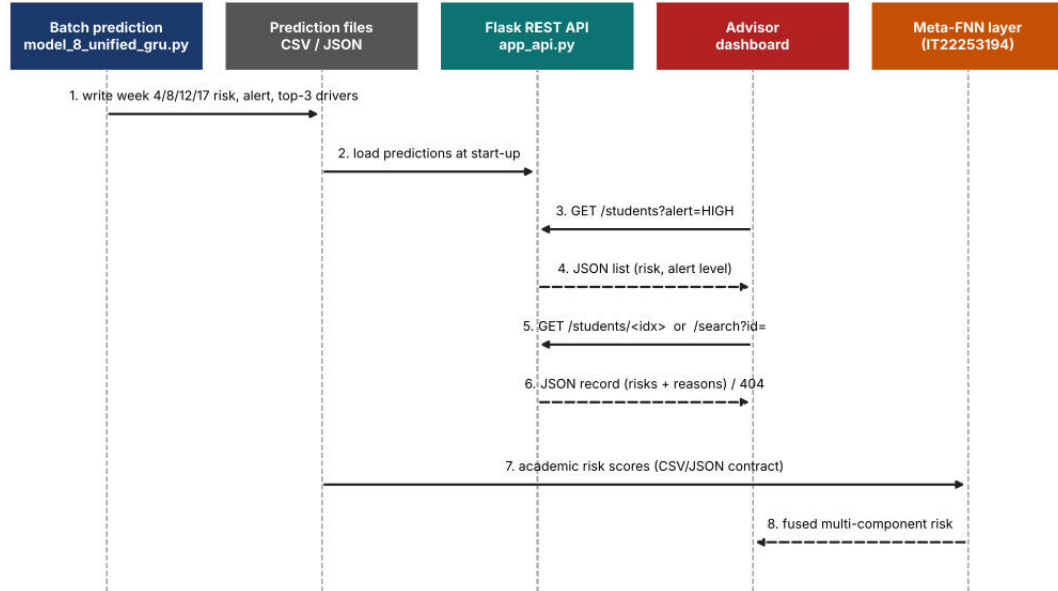


Figure 5.3: Sequence of data flow between the prediction component, API, dashboard and meta-layer

5.4 Verification and software testing

The component was verified at three levels: unit tests on the processed data and model outputs, integration tests on the Flask API and the meta-layer data contract, and user acceptance tests (UAT) on the advisor dashboard. The test cases and their results are listed in Table 5.3.

Table 5.3: Test cases

ID	Type	Test	Input	Expected result	Status
TC-01	Unit	Dataset shape	X_sequences.npy, y_labels.npy	16,950 × 17 × 13; 16,950 labels	Pass
TC	Unit	Missing	X_sequences.npy	No NaN	Pass

-02		feature values		values	
TC-03	Unit	Label encoding	y_labels.npy	Only values 0 and 1	Pass
TC-04	Unit	Test-set size	model8_predictions.csv	2,543 rows	Pass
TC-05	Unit	Missing output values	model8_predictions.csv	No null values	Pass
TC-06	Unit	Probability range	Week 4-17 risk columns	All values in [0, 1]	Pass
TC-07	Unit	Final risk consistency	academic_risk vs week17_risk	Identical for every student	Pass
TC-08	Unit	Unique identifiers	student_id	2,543 unique IDs	Pass
TC-09	Unit	Alert level mapping	alert_level vs academic_risk	LOW < 0.40, MEDIUM 0.40–0.70, HIGH ≥ 0.70	Pass
TC-10	Unit	Risk-driver ranking	top_feature_1–3 percentages	Descending order of contribution	Pass

TC -11	Unit	Week 17 reproducibility	Predictions, threshold 0.55	F1 = 0.7227, AUC = 0.8831	Pass
TC -12	Integration	CSV-JSON data contract	CSV and JSON exports	Same 2,543 records and values	Pass
TC -13	Integration	Health check	GET /health	HTTP 200; 2,543 predictions loaded	Pass
TC -14	Integration	Alert filter	GET /students?alert=HIGH	HTTP 200; 569 students	Pass
TC -15	Integration	Valid student	GET /students/0	HTTP 200; JSON record	Pass
TC -16	Integration	Invalid index	GET /students/99999	HTTP 404	Pass
TC -17	Integration	Search by ID	GET /search?id=OULAD_TEST_000	HTTP 200; JSON record	Pass
TC -18	Integration	Missing / unknown ID	GET /search; GET /search?id=NOPE	HTTP 400 / HTTP 404	Pass

TC -19	Integratio n	Summary	GET /summary	HTTP 200; alert- level counts	Pass
TC -20	UAT	Dashboard high-risk view	Open dashboard, filter HIGH	Risk cards, alerts and drivers shown (Figure 5.2)	Pass

5.5 Deployment and operation

The component uses batch inference rather than live per-request prediction. After the weekly feature update, the trained model is run once over all students and the results are exported as CSV and JSON files. The Flask API (app_api.py, port 5001) loads these files at start-up and serves them to the dashboard, and the prediction files are passed to the fusion meta-layer. This mirrors operational early warning systems such as OU Analyse [20], which also produce scheduled weekly predictions. The operational workflow is summarised in Table 5.4.

Table 5.4: Operational workflow

Step	Action	Output
1	Update weekly features from LMS logs	X_sequences.npy
2	Run batch prediction with the trained Unified GRU	model8_predictions.csv / .json
3	Start the Flask API (python app_api.py)	Endpoints on port 5001

4	Open the advisor dashboard in a browser	Risk cards, alerts and drivers
5	Send the prediction files to the fusion meta-layer	Combined student risk

6 RESULTS AND VALIDATION

6.1 Research results

Table 6.1: Performance at each intervention window (test set, n = 2,543)

Window	F1	AUC	Precision	Recall
Week 4	0.5178	0.6876	0.4078	0.7093
Week 8	0.6081	0.7813	0.5410	0.6942
Week 12	0.6352	0.8110	0.5703	0.7168
Week 17	0.7227	0.8831	0.7236	0.7218

Table 6.1 shows that F1 and AUC rise steadily across windows (Figure 6.1). AUC passes 0.75 at Week 8, when 9 to 22 weeks of course time remain, answering RQ1.

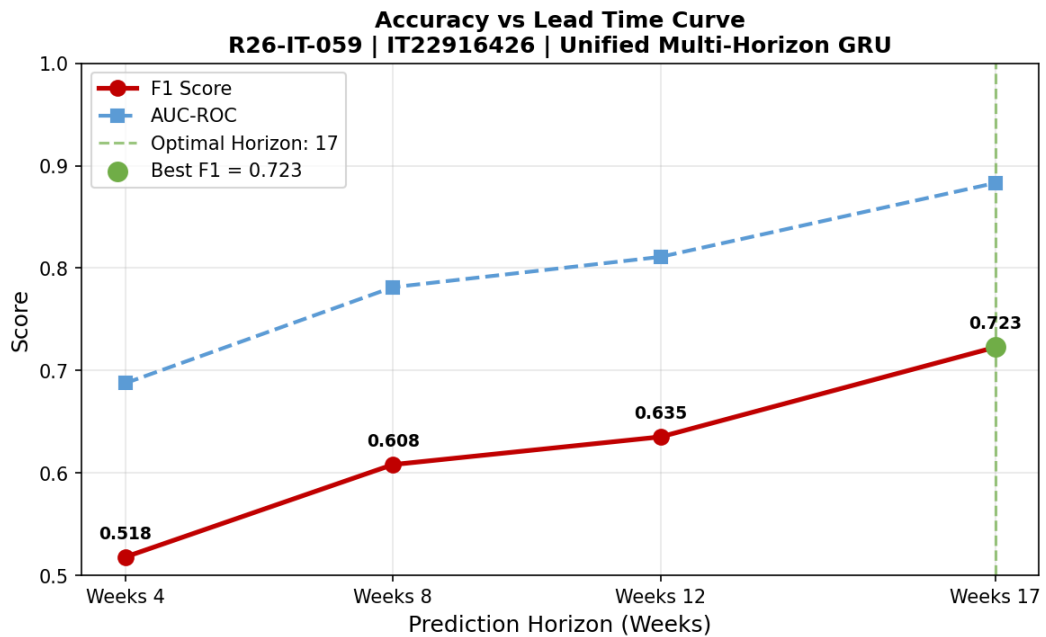


Figure 6.1: Accuracy versus lead time

Table 6.2: Comparison with baselines (Week 17)

Model	F1	AUC	Precision	Recall
Random	0.6647	0.8735	0.8226	0.5576

Forest				
Logistic Regression	0.6805	0.8694	0.7920	0.5965
XGBoost	0.6890	0.8772	0.7974	0.6065
Gradient Boosting	0.6965	0.8776	0.8046	0.6140
Simple LSTM	0.7087	0.8785	0.7070	0.7105
BiLSTM	0.7107	0.8757	0.7213	0.7005
Single-task GRU	0.7185	0.8788	0.6828	0.7581
Proposed	0.7227	0.8831	0.7236	0.7218

Table 6.2 and Figure 6.2 show that the proposed model gives the best and most balanced result; McNemar's test confirms the improvement over the strongest baseline ($\chi^2 = 6.674$, $p = 0.0098$), answering RQ2.

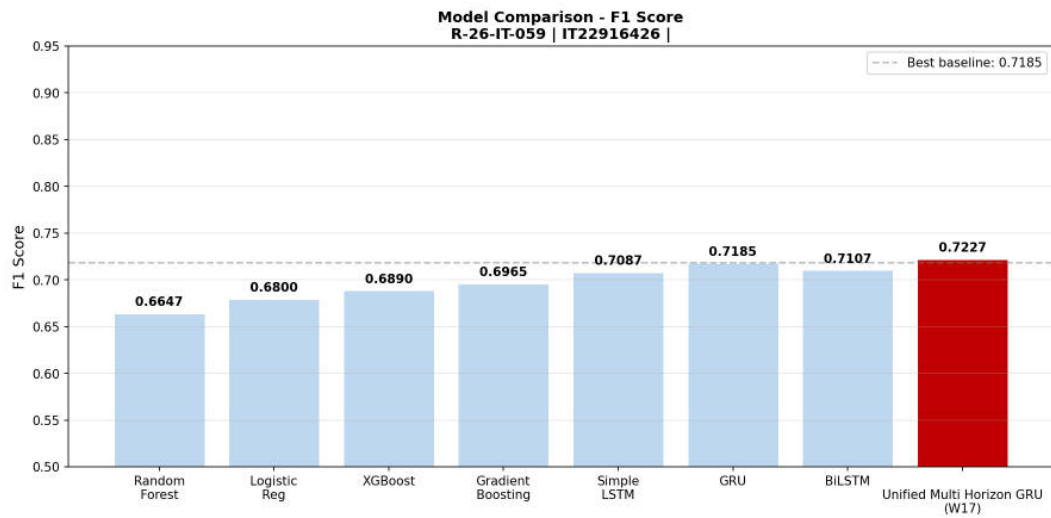


Figure 6.2: Model comparison

Explainability (RQ3): attention is highest in Weeks 5–9 (Figure 6.3). login_count (20.2%), inactive_days (12.8%), num_submitted (12.7%) and weeks_since_active

(9.1%) dominate; engagement features account for about 66% of importance (Figure 6.4). Figure 6.5 shows a per-student explanation.

SHAP DeepExplainer confirmed the same top three features (login_count 25.2%, inactive_days 13.1%, num_submitted 12.6%). An ablation variant using causal attention with a gate gave lower performance (F1 0.7122, AUC 0.8816), so the original architecture was kept.

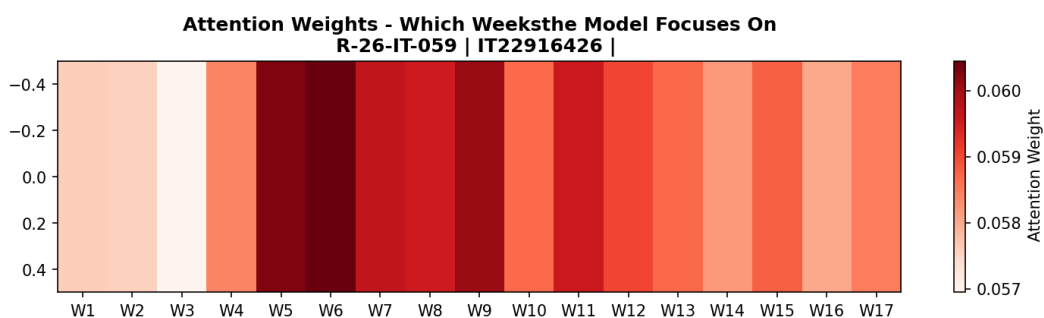


Figure 6.3: Temporal attention weights

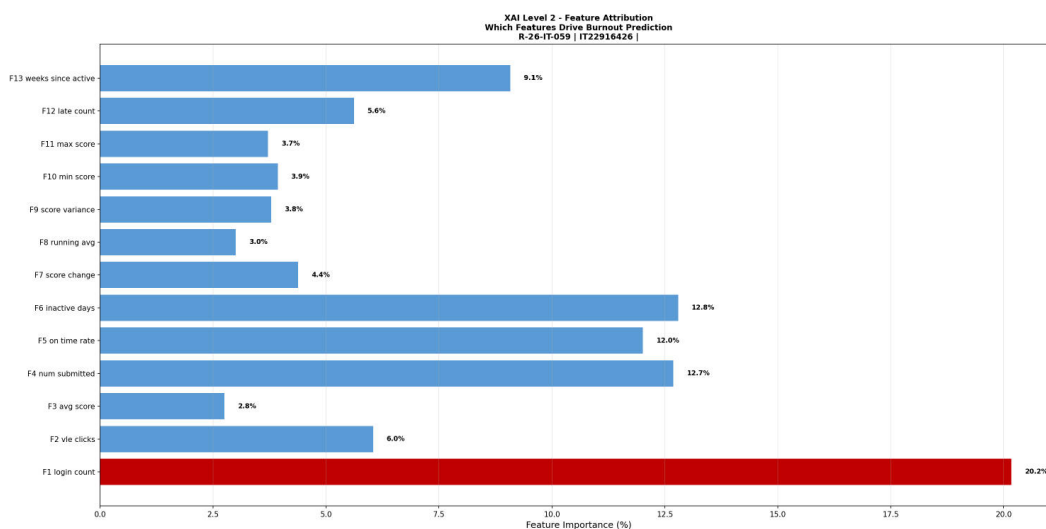


Figure 6.4: Global feature attribution

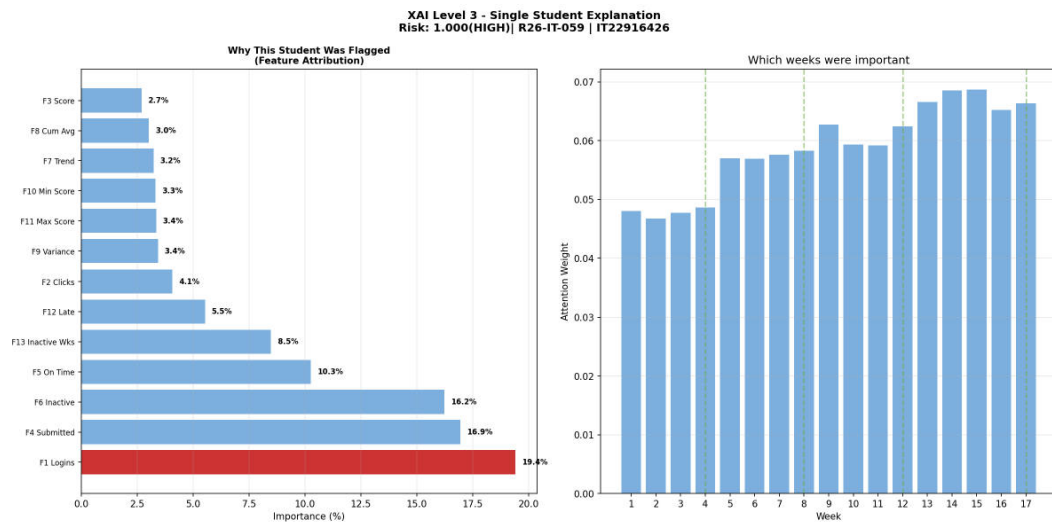


Figure 6.5: Per-student explanation for a high-risk student

6.2 Product and technical results

The implemented component met its functional and non-functional requirements. All 20 test cases in Table 5.3 passed (11 unit, 8 integration and 1 user acceptance test). Integration testing found one defect: the single-student endpoints returned HTTP 500 because NumPy integer values were not JSON-serialisable. The `clean_row()` function was corrected to convert them to native Python types, after which all endpoints responded correctly. Response times were measured with curl over three requests per endpoint. The slowest request, the full list of 2,543 students, took 0.09 s, and all other endpoints responded in under 0.04 s, satisfying NFR3. The dashboard displayed predictions for all 2,543 test students (Figure 5.1) and filtered correctly by alert level (Figure 5.2).

6.3 User or stakeholder validation

No formal user study with student advisors was conducted within the project timeline. The dashboard and API were demonstrated to the project supervisors during the progress presentation (PP2) in September 2026. The system has therefore been validated technically (Section 6.2) but not yet with its intended users. A controlled study measuring whether explanation-paired alerts improve advisor decisions is identified as future work (Section 9.4).

6.4 Integration results

Table 6.3: Generalisation validation (Week 17)

Protocol	n	F1	AUC
Standard held-out	2,543	0.7227	0.8831
Cross-year (2013→2014)	9,336	0.6332	0.8275
Concurrent- student (max-pool, exploratory)	839	0.7464	0.7615

In the cross-year protocol, the model was trained on the 2013 presentations (7,614 students) and tested on the 2014 presentations (9,336 students), with normalisation fitted on the training year only. At Week 17 it reached F1 0.6332 and AUC 0.8275, a drop of 0.056 in AUC from the standard held-out result. However, the earlier windows did not reach the 0.75 threshold (AUC 0.6405 at Week 4, 0.6694 at Week 8 and 0.7235 at Week 12). Generalisation to a new cohort therefore holds for end-of-course prediction, but early alerts would need retraining or calibration for each new cohort.

For concurrently enrolled students, count-based aggregation reached precision 0.9924 but recall only 0.2253 (F1 0.3672), while max-pool aggregation raised F1 to 0.7464 (Figure 6.6). These results are exploratory only: the 839 students were drawn from the full dataset rather than the held-out test split, and the decision threshold was set on the same data. They show that max-pool aggregation avoids recall collapse, but they are not an unbiased performance estimate.

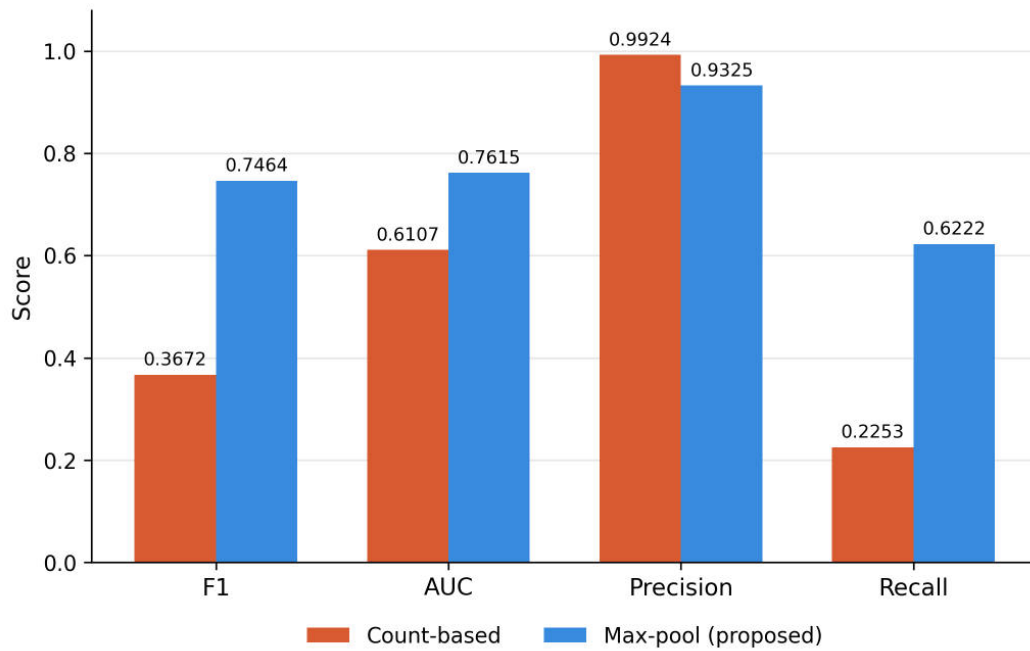


Figure 6.6: Count-based vs max-pool aggregation

Integration with the fusion meta-layer was verified through the shared CSV/JSON data contract (Listing 5.5). The prediction file for all 2,543 test students was consumed by the Meta-FNN layer (IT22253194) without format errors.

6.5 Summary of findings

- Risk is reliably detectable from Week 8 (AUC 0.78).
- Week 17 gives the best performance (F1 0.7227, AUC 0.8831), significantly better than baselines.
- Engagement signals, especially login frequency, drive risk.
- Cross-year generalisation holds at Week 17 (AUC 0.8275), but earlier windows fall below 0.75; concurrent-student results are exploratory.

7 DISCUSSION

7.1 Interpretation of findings

The results distinguish the earliest reliable alert (Week 8, AUC 0.78) from the most accurate prediction (Week 17, AUC 0.88), giving institutions practical guidance on when to act. Engagement features accounted for about 66% of attribution, suggesting that reduced VLE activity is an early signal of risk, so detailed activity logging is more informative than grade tracking alone. Generalisation was only partly confirmed (RQ4): on a new cohort, Week 17 remained strong (AUC 0.83), but earlier windows fell below 0.75, so early alerts may need recalibration for each cohort.

7.2 Comparison with literature baselines and existing solutions

Like Akçapınar et al. [3], accuracy rises over time, but here within a single model. Unlike global SHAP studies [1], [13], explanations are given per student and per horizon, addressing the actionability gap identified by Nagy and Molontay [4].

On the same data, the proposed model outperformed classical baselines such as Random Forest (F1 0.6647) by a clear margin, but its advantage over sequential deep models was small (F1 +0.0042 over the single-task GRU), consistent with reports that recurrent models perform similarly on VLE data [1], [7].

7.3 Research contribution

- An empirical accuracy-versus-lead-time curve from one unified model.
- A three-level, advisor-facing explainability framework.
- Multi-protocol validation, including a cross-year test and an exploratory analysis of concurrently enrolled students.

7.4 Product contribution

A working prediction API and advisor dashboard that integrate with the team's meta-layer and present reason-based alerts.

7.5 Limitations and threats to validity

- The 17-week filter excludes early withdrawals (at-risk share falls from 52.8% to 31.4%).
- Results are based on a UK distance-learning dataset; transfer to Sri Lankan face-to-face settings is untested.
- The F1 gain over the single-task GRU is small (+0.0042), though the model adds multi-horizon and explainability capability.
- Withdrawal and failure are merged into one label.
- No live advisor study has yet measured impact on intervention behaviour.
- Cross-year generalisation holds only at Week 17; earlier windows fall below AUC 0.75 on a new cohort.
- The concurrent-student analysis is exploratory, because students were not drawn from the held-out split and the threshold was set on the same data.
- SHAP DeepExplainer does not fully support GRU layers, so SHAP values are approximate; gradient attribution was used as the primary method, and the two agree on the top three features.

7.6 Implications

Institutions should log VLE engagement in detail, trigger first alerts around Week 8, and pair each alert with its explanation so advisors can choose targeted actions such as a check-in, peer mentoring or a skills workshop.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Target users are universities, online learning providers and LMS vendors. The value proposition is earlier, explainable identification of at-risk students, reducing attrition and the associated loss of tuition revenue and student wellbeing.

User persona – Student advisor. Nimali is an academic advisor responsible for around 300 students in an online degree programme. She has no time to review every student's LMS activity and currently learns about struggling students only after they miss assessments. She needs a weekly list showing who is at risk, how urgent the risk is, and why, so she can choose the right action (a check-in, peer mentoring or study-skills support) before the student withdraws.

8.2 Market and competitive position

Existing analytics tools mostly provide single-point or rule-based flags. This component differentiates through multi-horizon timing guidance and per-student explanations.

Table 8.1: Comparison with existing solutions

Feature	Moodle Analytics [19]	OU Analyse [20]	Commercial Platforms (e.g. EAB Navigate, Civitas Learning)	Unified Multi-Horizon GRU
Risk prediction	Yes (binary at-risk)	Yes (weekly)	Yes (proprietary risk scores)	Yes, at 4 horizons
Prediction timing	Chosen time splits	Weekly	Varies by vendor	Weeks 4, 8, 12, 17 with earliest reliable point

				identified
Model	Engagement indicators (logistic regression / neural network)	Machine learning on demographic and VLE data	Proprietary	Unified multi-horizon GRU
Per-student explanation	No (at-risk flag only)	Limited	Limited / not published	Yes, top three risk drivers per student
Cost / access	Free, open source	Internal to the Open University	Paid licence	Planned low-cost plugin

8.3 Business or sustainability model

A tiered, subscription-based model is proposed, priced per enrolled student per year so that cost grows with the size of each institution. Because inference runs in weekly batches, computing cost is low: a single small cloud server can serve thousands of students, and retraining takes only minutes on a GPU. Table 8.2 describes the proposed service packages. All prices are indicative estimates for the Sri Lankan market and would be validated through institutional pilots.

Table 8.2: Proposed service packages (indicative pricing)

Package	Target customer	Features included	Indicative price (LKR)
Pilot (free)	A single programme or module trialling the system	Week 17 risk list for one course (up to 300 students); alert levels; monthly CSV export; email support	Free for one semester
Standard	Faculties and small private higher-education institutes	All four horizons (Weeks 4, 8, 12, 17); per-student explanations (top risk drivers); advisor dashboard; weekly batch updates; CSV/JSON export	150 per enrolled student per year

Professional	Universities with student-support units	Everything in Standard, plus integration with the R26-IT-059 meta-layer (emotional and behavioural signals), REST API access, LMS (Moodle) plugin, retraining on the institution's data each semester and advisor training	250 per enrolled student per year
Enterprise / LMS partner	LMS vendors and multi-campus institutions	White-label licence, on-premise deployment option, service-level agreement, custom features and dedicated support	Negotiated annual licence

Table 8.3 estimates the annual operating cost of the service, based on cloud hosting, periodic retraining and part-time support.

Table 8.3: Estimated annual operating cost (indicative)

Cost item	Assumption	Annual cost (LKR)
Cloud server and storage	One small virtual machine running the API, dashboard and weekly batch jobs	180,000
GPU time for retraining	Retraining per institution each semester on a rented GPU	60,000
Maintenance and support	Part-time developer for updates, monitoring and customer support	360,000
Total		600,000

On these assumptions, the Standard package breaks even at about 4,000 enrolled students ($600,000 \div 150$), which is roughly one medium-sized faculty, or two faculties on the Professional package. The go-to-market plan is to start with a free pilot at SLIIT, use its results as evidence, expand to other Sri Lankan state and private universities through the Standard and Professional packages, and later offer an LMS plugin to reach institutions abroad. Revenue would fund hosting, retraining and continued development, keeping the service sustainable after the research project ends.

8.4 Scalability intellectual property and compliance

The model has only 767,493 parameters and uses batch inference, so it can be retrained on each institution's data and served from inexpensive hardware. In Sri Lanka, the Personal Data Protection Act No. 9 of 2022 became fully enforceable on 18 March 2025, when Parts I, II, III and VII came into operation [21]. Any deployment would therefore need a lawful basis for processing student activity data, transparency towards students, data minimisation, and appropriate security safeguards. GDPR would apply to clients in the European Union. The source code is the author's own work; OULAD is used under its CC-BY 4.0 licence [14].

8.5 Contribution to the Sustainable Development Goals

The component supports SDG 4 (Quality Education), particularly target 4.3 on equal access to tertiary education, by helping institutions retain students who might otherwise drop out.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This component delivers an explainable, multi-horizon dropout risk predictor that tells advisors when to act and why. It identifies Week 8 as the earliest reliable alert point and Week 17 as optimal, outperforms seven baselines, and generalises to a new cohort at Week 17.

9.2 Achievement of objectives

Table 9.1 summarises how each research objective was achieved and where the evidence is reported.

Table 9.1: Achievement of objectives

Objective	Status	Evidence
O1 Features	Achieved	Section 3.3
O2 Unified GRU	Achieved	Section 4.4
O3 Explainability	Achieved	Section 6.1
O4 Benchmarking	Achieved	Table 6.2
O5 Generalisation	Partly achieved	Table 6.3
O6 API and integration	Achieved	Section 5.3

9.3 Individual research and product contributions

The research contributions of this component were prepared for publication in two venues:

- A full paper was submitted to the 17th International Learning Analytics and Knowledge Conference (LAK 2027); the notification of acceptance is expected on 24 November 2026.
- A journal manuscript was prepared for the Journal of Computer Assisted Learning (JCAL, Wiley), on the recommendation of the supervisor. JCAL covers the use of information and communication technologies to support

learning, teaching and instructional design, which matches the learning-analytics focus of this work.

The product contributions are the Flask prediction API, the advisor dashboard and the CSV/JSON integration contract with the team's fusion meta-layer.

9.4 Future directions

- Variable-length sequence models to include early withdrawals.
- Multi-class outcomes separating withdrawal from failure.
- Validation on SLIIT or other Sri Lankan LMS data.
- A controlled advisor study measuring the effect of explanation-paired alerts.
- Fairness auditing across demographic groups.

REFERENCES

- [1] H. Waheed et al., "Early prediction of learners at risk in self-paced education: A neural network approach," *Expert Syst. Appl.*, vol. 213, p. 118868, 2023, doi: 10.1016/j.eswa.2022.118868.
- [2] B. Alnasyan, M. Basher, and M. Alassafi, "The power of deep learning techniques for predicting student performance in virtual learning environments: A systematic literature review," *Comput. Educ.: Artif. Intell.*, vol. 6, p. 100231, 2024, doi: 10.1016/j.caeai.2024.100231.
- [3] G. Akçapınar, A. Altun, and P. Aşkar, "Using learning analytics to develop early-warning system for at-risk students," *Int. J. Educ. Technol. High. Educ.*, vol. 16, p. 40, 2019, doi: 10.1186/s41239-019-0172-z.
- [4] M. Nagy and R. Molontay, "Interpretable dropout prediction: Towards XAI-based personalized intervention," *Int. J. Artif. Intell. Educ.*, vol. 34, pp. 274–300, 2024, doi: 10.1007/s40593-023-00331-8.
- [5] S. Boujmiraz, H. Darhmaoui, and A. D. El Maliani, "Predicting student performance: A comprehensive review of machine learning, deep learning, and explainable AI approaches," *Comput. Educ.: Artif. Intell.*, p. 100548, 2026, doi: 10.1016/j.caeai.2026.100548.
- [6] F. Jimenez, A. Paoletti, G. Sanchez, and G. Sciavicco, "Predicting the risk of academic dropout with temporal multiobjective optimization," *IEEE Trans. Learn. Technol.*, vol. 12, no. 2, pp. 225–235, 2019, doi: 10.1109/TLT.2019.2911070.
- [7] W. Torkhani and K. Rezgui, "OULAD MOOC student performance prediction using machine and deep learning," in *Proc. ICODAI 2024, Atlantis Highlights Intell. Syst.*, vol. 12, pp. 229–241, 2025, doi: 10.2991/978-94-6463-654-3_18.
- [8] A. Villar and C. R. V. de Andrade, "Supervised machine learning algorithms for predicting student dropout and academic success: A comparative study," *Discov. Artif. Intell.*, vol. 4, p. 2, 2024, doi: 10.1007/s44163-023-00079-z.

- [9] A. M. Rabelo and L. E. Zarate, "A model for predicting dropout of higher education students," *Data Sci. Manag.*, vol. 8, pp. 72–85, 2025, doi: 10.1016/j.dsm.2024.07.001.
- [10] M. Rebelo Marcolino et al., "Student dropout prediction through machine learning optimization: Insights from Moodle log data," *Sci. Rep.*, vol. 15, p. 9840, 2025, doi: 10.1038/s41598-025-93918-1.
- [11] M. Vaarma and H. Li, "Predicting student dropouts with machine learning: An empirical study in Finnish higher education," *Technol. Soc.*, vol. 76, p. 102474, 2024, doi: 10.1016/j.techsoc.2024.102474.
- [12] Y.-H. Chang, F.-C. Chen, and C.-I. Lee, "Developing an early warning system with personalized interventions to enhance academic outcomes for at-risk students in Taiwanese higher education," *Educ. Sci.*, vol. 15, p. 1321, 2025, doi: 10.3390/educsci15101321.
- [13] S. Mustofa, Y. R. Emon, S. B. Mamun, S. A. Akhy, and M. T. Ahad, "A novel AI-driven model for student dropout risk analysis with explainable AI insights," *Comput. Educ.: Artif. Intell.*, vol. 8, p. 100352, 2025, doi: 10.1016/j.caeai.2024.100352.
- [14] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics dataset," *Sci. Data*, vol. 4, p. 170171, 2017, doi: 10.1038/sdata.2017.171.
- [15] E. Ofori and D. K. Dake, "Explainable artificial intelligence in LSTM transformer models for student performance analysis," *Discov. Comput.*, vol. 28, p. 313, 2025, doi: 10.1007/s10791-025-09814-9.
- [16] N. U. R. Junejo, M. W. Nawaz, Q. Huang, X. Dong, C. Wang, and G. Zheng, "Accurate multi-category student performance forecasting at early stages of online education using neural networks," *Sci. Rep.*, vol. 15, p. 16251, 2025, doi: 10.1038/s41598-025-00256-3.
- [17] B. Alnasyan, M. Basher, M. Allassafi, and K. Alnasyan, "Kanformer: An attention-enhanced deep learning model for predicting student performance in virtual

learning environments," *Soc. Netw. Anal. Min.*, vol. 15, p. 25, 2025, doi: 10.1007/s13278-025-01446-7.

[18] K. O. Adefemi, M. B. Mutanga, and V. Jugoo, "Hybrid deep learning models for predicting student academic performance," *Math. Comput. Appl.*, vol. 30, no. 3, p. 59, 2025, doi: 10.3390/mca30030059.

[19] Moodle, "Students at risk of dropping out," Moodle Docs. [Online]. Available: https://docs.moodle.org/en/Students_at_risk_of_dropping_out

[20] C. Herodotou, M. Hlosta, A. Boroowa, B. Rienties, Z. Zdrahal, and C. Mangafa, "Empowering online teachers through predictive learning analytics," *Br. J. Educ. Technol.*, vol. 50, no. 6, pp. 3064–3079, 2019, doi: 10.1111/bjet.12853.

[21] Data Protection Authority of Sri Lanka, "Personal Data Protection Act dates of operation announced." [Online]. Available: <https://dpa.gov.lk/est.php>

[22] Q. Huang and J. Chen, "Enhancing academic performance prediction with temporal graph networks for massive open online courses," *J. Big Data*, vol. 11, p. 52, 2024, doi: 10.1186/s40537-024-00918-5.

[23] K. Niu, G. Lu, X. Peng, Y. Zhou, J. Zeng, and K. Zhang, "CNN autoencoders and LSTM-based reduced order model for student dropout prediction," *Neural Comput. Appl.*, vol. 35, no. 30, pp. 22341–22357, 2023, doi: 10.1007/s00521-023-08894-2.

[24] A. Kukkar, R. Mohana, A. Sharma, and A. Nayyar, "A novel methodology using RNN + LSTM + ML for predicting student's academic performance," *Educ. Inf. Technol.*, vol. 29, pp. 14365–14401, 2024, doi: 10.1007/s10639-023-12394-0.

[25] E. Yeskuatov, L. K. Foo, and S.-L. Chua, "Detecting burnout among undergraduate computing students with supervised machine learning," *Healthcare*, vol. 13, p. 3182, 2025, doi: 10.3390/healthcare13233182.

[26] L. Jin, Y. Wang, H. Song, and H.-J. So, "Predictive modelling with the Open University Learning Analytics Dataset (OULAD): A systematic literature review," in *Artificial Intelligence in Education: AIED 2024 Workshops, Commun. Comput. Inf. Sci.*, vol. 2150, pp. 477–484, 2024, doi: 10.1007/978-3-031-64315-6_46.

- [22] Q. Huang and J. Chen, "Enhancing academic performance prediction with temporal graph networks for massive open online courses," *J. Big Data*, vol. 11, p. 52, 2024, doi: 10.1186/s40537-024-00918-5.
- [23] K. Niu, G. Lu, X. Peng, Y. Zhou, J. Zeng, and K. Zhang, "CNN autoencoders and LSTM-based reduced order model for student dropout prediction," *Neural Comput. Appl.*, vol. 35, no. 30, pp. 22341–22357, 2023, doi: 10.1007/s00521-023-08894-2.
- [24] A. Kukkar, R. Mohana, A. Sharma, and A. Nayyar, "A novel methodology using RNN + LSTM + ML for predicting student's academic performance," *Educ. Inf. Technol.*, vol. 29, pp. 14365–14401, 2024, doi: 10.1007/s10639-023-12394-0.
- [25] E. Yeskuatov, L. K. Foo, and S.-L. Chua, "Detecting burnout among undergraduate computing students with supervised machine learning," *Healthcare*, vol. 13, p. 3182, 2025, doi: 10.3390/healthcare13233182.
- [26] L. Jin, Y. Wang, H. Song, and H.-J. So, "Predictive modelling with the Open University Learning Analytics Dataset (OULAD): A systematic literature review," in *Artificial Intelligence in Education: AIED 2024 Workshops*, *Commun. Comput. Inf. Sci.*, vol. 2150, pp. 477–484, 2024, doi: 10.1007/978-3-031-64315-6_46.

GLOSSARY

Ablation study - An experiment that removes or changes one part of a model to measure its effect on performance.

At-risk student - A student whose final result was Withdrawn or Fail, labelled 1 in this study.

Batch inference - Running a trained model once over all students on a schedule, rather than on each individual request.

Early stopping - Stopping training when validation performance has not improved for a set number of epochs, to avoid overfitting.

Gradient attribution - An explainability method that measures how much each input feature changes the model output, using the model's gradients.

Intervention window - A week at which a prediction is made (Weeks 4, 8, 12 and 17) so that advisors can act.

Max-pool aggregation - Combining a student's risks across several modules by taking the highest module risk.

Multi-horizon prediction - Predicting the same outcome at several points in time from one model.

Survivorship effect - Bias caused by keeping only cases that lasted long enough to be observed, here students with at least 17 weeks of activity.

Temporal attention - A layer that assigns a weight to each week of the sequence to indicate its importance.

Z-score normalisation - Rescaling each feature to zero mean and unit standard deviation using training-set statistics.

APPENDICES

Appendix A Extended cross-year results

Table A.1 reports the cross-year validation results (trained on 2013, tested on 2014, $n = 9,336$) at every intervention window, extending Table 6.3.

Table A.1: Cross-year validation results by intervention window

Window	F1	AUC	Precision	Recall
Week 4	0.4868	0.6405	0.3992	0.6237
Week 8	0.5237	0.6694	0.4220	0.6901
Week 12	0.5531	0.7235	0.4454	0.7294
Week 17	0.6332	0.8275	0.5392	0.7669

Appendix B Academic integrity and AI use disclosure

Claude (Anthropic) was used as an assistant during the preparation of this thesis for the following purposes:

- Drafting an initial chapter skeleton from the author's own manuscripts (LAK 2027 paper and JCAL draft).
- Checking grammar, British spelling and IEEE reference formatting.
- Debugging code, including identifying the scaler leakage in cross-year validation and the JSON serialisation defect in the Flask API.
- Running the verification tests in Table 5.3 against the author's own data files and code.
- Drawing Figures 4.1 and 5.3 from the author's system design.

All models, experiments and results were designed, run and produced by the author using the author's own code. All AI-assisted text was reviewed, rewritten and verified by the author, who takes full responsibility for the content.